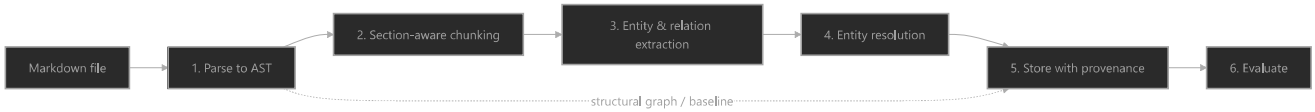
From Markdown to a Knowledge Graph: Pipeline Summary

This guide covers what happens **after** data has been extracted from a document link into Markdown: turning that Markdown into nodes and edges in a knowledge graph (KG).



Step 0: Decide the purpose and schema first

Before writing code, answer three questions: **what will the graph be used for** (question answering / GraphRAG, exploring cross-document links, analytics)? **Which node types** do you need? **Which relation types** do you need?

A small fixed ontology (roughly 5–10 node types and a short list of relation types) produces a clean, queryable graph. A schema-free graph is faster to start but becomes hard to query, because the same relation appears under many names ("uses", "utilizes", "is built with").

Step 1: Parse the Markdown into a tree

Parse the file into an abstract syntax tree (AST) instead of treating it as plain text. Useful libraries are `markdown-it-py` or `mistune` in Python and `remark` in JavaScript.

The document's structure already gives you part of the graph:

Markdown element	Graph mapping
The document itself	`(:Document {title, source_url})`
`#` / `##` headings	`(:Section)` nodes linked by `PART_OF` edges
`[text](url)` links	`REFERENCES` edges to other documents or entities
Tables	Rows become entities; columns become attributes or relations
Lists under a heading	Often "X has members/features Y, Z" relations

This structure-only graph is your baseline. It is deterministic, cheap, and accurate. Every later step should be shown to add value over it.

Step 2: Chunk by section and keep context

Split text at heading boundaries rather than at fixed token counts. Attach the heading path to each chunk (e.g. `Methods > Data Collection`). A sentence like "It uses 3 sensors" cannot be interpreted without knowing what "it" refers to.

Step 3: Extract entities and relations

Approach	Strengths	Weaknesses
Rule / NLP-based (spaCy NER, dependency patterns)	Fast, free, reproducible	Brittle; only generic entity types
Schema-guided LLM extraction	Flexible; follows your ontology; current common practice	Cost, non-determinism, can hallucinate relations
Open IE / schema-free	High recall	Messy, inconsistent relation names

For schema-guided LLM extraction, give the model the allowed node and edge types and ask for JSON triples:

```
{"head": "CropWatch", "head_type": "Project", "relation": "USES", "tail": "Neo4j", "tail_type": "T
```

Libraries that wrap this pattern include LangChain's `LLMGraphTransformer`, LlamaIndex's `PropertyGraphIndex`, and Microsoft's GraphRAG. Check their current documentation, since APIs change often.

Step 4: Resolve entities (the most underestimated step)

Different mentions of the same thing must collapse into one node. For example, "Graphic Era University", "GEU", and "the university" should be a single entity. Otherwise the graph fragments into duplicates and multi-hop queries fail silently.

Common techniques are string normalization (case, punctuation, abbreviations), embedding similarity with a tuned threshold, and LLM-assisted merge decisions for ambiguous cases. Most quality loss in practice happens here.

Step 5: Store the graph with provenance

Every triple should record where it came from: document ID, chunk ID, and ideally the text span. Provenance lets you debug wrong edges and show users evidence for an answer.

Store	Query language	Best when
Neo4j (property graph)	Cypher	Getting started quickly; application-style queries
RDF triple store	SPARQL	Standard ontologies and linked-data interoperability

matter

Use `MERGE` instead of `CREATE` in Neo4j so that re-ingesting a document does not create duplicates:

```
MERGE (p:Project {name: "CropWatch"})
MERGE (t:Tool {name: "Neo4j"})
MERGE (p)-[:USES {source_chunk: "doc1#architecture"}]->(t)
```

Step 6: Evaluate

1. **Build a gold set:** hand-annotate the correct triples for 5–10 representative documents.
2. **Measure:** precision and recall of extracted triples against the gold set.
3. **Compare:** against the structure-only baseline from Step 1.
4. **Analyse failures:**
   - hallucinated relations (facts inferred but not stated in the text)
   - duplicate entities that were not merged
   - relations lost across chunk boundaries
   - inconsistent relation naming

Worked example

Input Markdown

```
## Architecture
CropWatch uses Neo4j for storage and is developed at Graphic Era.
```

Extracted triples

Head	Relation	Tail
CropWatch (Project)	USES	Neo4j (Tool)
CropWatch (Project)	DEVELOPED_AT	Graphic Era (Organization)
Architecture (Section)	PART_OF	Document

Suggested learning path

1. Build Steps 1 and 5 only: a structure-only graph loaded into Neo4j.
2. Add LLM extraction (Step 3) for one or two node types.
3. Add entity resolution (Step 4) and observe how the graph changes.
4. Run the evaluation (Step 6) and compare against the baseline.

Checklist

- ⬛ Purpose and schema defined
- ⬛ Markdown parsed into an AST
- ⬛ Structural baseline graph built